Laboratree — Foundation Plan (v1)

The Trustworthy Agentic Research Lab

Context

Laboratree is a greenfield SaaS: an **end-to-end, multi-agent research lab** for the full lifecycle of primary/secondary data research. It is a **multi-agent orchestration platform with human-in-the-loop (HITL)** — some stages are AI-led (automated data science), some are human-led / AI-assisted (ideation, primary data collection). The directory is empty today (only `logo/logo.PNG`).

Positioning: not "another AutoML that spits out numbers," but the *trustworthy, reproducible, human-steered* research lab — winning where autonomous agents are documented to fail (hallucinated metrics, data leakage, unreliable self-eval). Serves **both** rigorous econometric/causal research **and** general ML on one shared trust layer.

> **v1 flagship narrative (goes DEEP):** *drop in any messy inputs → get clean consolidated data → understand a research paper in plain language → reproduce and then out-explore that paper — with every number provably backed by a re-runnable execution.* Concretely: `Signal Lab → Paper Lab (Study) → Paper Lab (Experiment)` on the trust layer. All other Labs are **scaffolded (breadth)**: interfaces + registry + ≥1 component + a page.

Decisions Locked With the User

- **Stack:** FastAPI (Python) API + Next.js/React frontend.
- **Persistence (polyglot, each in its own dedicated container): Postgres** (+pgvector), **Redis**, **Neo4j**, **MongoDB** — roles assigned below. **Blob storage: local filesystem for now** (mounted volume) behind a `BlobStore` interface; swap to S3/MinIO/GCS later with no call-site changes.
- **Orchestration: LangGraph** (v0.4+) — native durable checkpointing, `interrupt` HITL, crash-resume, time-travel. (Chosen over AutoGen, which entered maintenance mode.)
- **Agent execution: Hybrid** — curated registered tools for production stages + a sandboxed Docker code-executor for the Modeling R&D loop (we build the executor; LangGraph has none).
- **HITL: Per-stage, configurable** via `interrupt`; human-led Labs default to per-artifact approval, automated Labs to plan + final.
- **LLMs (the agents' brains): OpenAI** — GPT-5.x reasoning model (id in config; ~GPT-5.4-class) + `text-embedding-3-small`, behind one pluggable `LLMClient` (swap providers later).
- **Analytical model coverage (full power of ML + DL + econometrics):** the Model Lab is a **model zoo** spanning families — **classical ML** (scikit-learn, XGBoost/LightGBM/CatBoost), **Deep Learning** (PyTorch: MLP/CNN/RNN-LSTM-GRU/Transformer/autoencoder, for tabular, sequence, NLP, vision), **econometrics** (statsmodels: AR/MA/ARIMA/VAR/VECM/Logit/Probit/GLM), **time-series** (sktime, Darts incl. DL forecasters N-BEATS/TFT/DeepAR, Prophet, pmdarima), and **anomaly/unsupervised** (PyOD, isolation forest, DL autoencoders). Every model is a registry Component; the R&D loop can select across families. DL is first-class (sandbox executor is **GPU-optional**).
- **Lab modularity + one web pipeline:** each Lab is an **isolated plug-in/plug-out module** (own graph, components, page) — and a **cross-Lab Pipeline Canvas** wires them into a single end-to-end research web pipeline.
- **v1 depth:** Paper-reproduction story deep; other Labs breadth-scaffolded.
- **Theme: Light forest** from the logo.

The Four Signature Differentiators (the moat — all four in v1)

Each targets a *documented* failure of existing autonomous DS agents:

1

Provenance-locked results (Evidence Ledger)

Every value/figure is an **Evidence** record `{value, run_id, code_hash, data_version, artifact_ref, ts}`; charts/reports refuse to show any claim not backed by a real, re-runnable Evidence record. Kills failure-masking. Cross-cutting.

2

Leakage Sentinel + reproducibility

An agent audits pipelines for **target / temporal / train-test** leakage; every Run carries a **reproducibility manifest** (data hash, seeds, env image digest, code) for one-click deterministic reproduce.

3

Co-Scientist Ideation

Google-Nature pattern (Generate → Reflect → Elo-rank → Evolve → Meta-review) hypothesis engine, human-steered. (Ideation Lab; scaffolded deeper post-v1.)

4

Adversarial Red-Team Critic

Independent agent (distinct persona/model from the Engineer) that stress-tests candidate models (perturbation, ablation, sensitivity, subgroup, counterfactual + leakage re-check) and **gates** acceptance.

Prior Art Adopted (Researched)

RD-Agent (R&D loop) · Google AI Co-Scientist (hypothesis tournament) · Data Interpreter/MetaGPT (graph decomposition) · AIDE/ML-Master/EvoDS (iterative refinement + skill learning) · Leakage & Reproducibility Crisis literature (motivates the trust layer).

Flagship Labs DEEP in v1

Signal Lab — Noise → Signal consolidation (the front door)

Drop in *any* mix of raw inputs (PDF, Word, Excel, CSV, images/scans). Agents **extract** → **clean** → **reconcile** → **consolidate** into one **master workbook** (single `.xlsx` with segregated sheets + a data dictionary). HITL confirms ambiguous field mappings.

- **Extraction stack (reuse):** `unstructured`, PyMuPDF/`pdfplumber`, `camelot`/`tabula-py` (tables), `python-docx`, `openpyxl`/`pandas`, `pytesseract` OCR fallback; LLM for schema inference, entity extraction, and cross-file reconciliation. Output feeds Data Lab / Paper Experiment.

Paper Lab — Study mode (understand any paper)

Upload a paper (PDF) → an agentic parse produces a plain-language **Paper Card**: Problem Statement (simple) · Model(s) used · Data Sources · Preprocessing funnel (short) · Data Sample · Independent variables + Target · Variants · **Math formulas with beginner-friendly explanations** · Results & Inference (simple).

- **"Explain simpler" button** — progressive disclosure: each press escalates a claim to a simpler level (add analogies, worked examples, plain re-statement). Levels cached per claim.
- **Chat-with-paper** — agentic RAG over the paper (pgvector) with citations to sections.

Paper Lab — Experiment mode (reproduce, then out-explore)

- **Auto data-fetch agent** — hunts the paper's cited datasets across known repositories (UCI, Kaggle, data.gov, World Bank, FRED, Zenodo, OSF, DOIs, direct links); retries with strategies. On failure after max tries it **stops and hands off to HITL** with the exact source link + guided manual-download-and-upload instructions (honest fallback, not a fake success).
- **Interactive walkthrough** — reconstruct the paper's pipeline as a **node graph** (data → preprocess → EDA → model → result → inference) on a React Flow canvas. Normal users get a **live walkthrough**; power users **fork any node** to try alternatives and **compare against the paper's reported results**. Nodes are powered by the *same registry components* as Data/Insight/Modeling, and every produced number is written to the **Evidence Ledger** — so "we beat the paper" is provable, and Red-Team/Leakage checks apply to the fork.

Scaffolded Labs breadth in v1; deepened later

Data (connectors + transforms; Leakage Sentinel hooks) · Insight (EDA + Vega-Lite charts; causal-discovery hints) · **Model Lab** (R&D loop graph; the full model zoo organized as pluggable families — ML / **DL (PyTorch)** / econometrics / time-series / anomaly; Red-Team gate) · Trend (decomposition + causal-impact) · Decision (rule/scenario + counterfactual) · Ideation (Co-Scientist) · Collection (survey/sampling/bias + synthetic pilot respondents) · Intelligence (agentic RAG + Evidence-bound report card).

A few real Data/Insight/Model components across ML **and** DL families are built in v1 because the Paper Experiment walkthrough must reproduce both ML and DL papers. All Labs are composable on the **cross-Lab Pipeline Canvas**.

Core Architecture

- **Agent roles (LangGraph nodes):** Orchestrator/Planner (goal→subtask DAG) · Researcher · Engineer (curated tools **or** sandboxed code) · **Red-Team Critic** · **Leakage Sentinel** · Reporter (Evidence-bound) · Human (`interrupt` = HITL). Each Lab assembles a `StateGraph`.
- **Modeling R&D loop:** hypothesis → code in Docker sandbox → execute → evaluate → Red-Team + Leakage → feedback → refine, selecting across the full **model zoo**. Each family is a registered Component group with task-appropriate evaluators; winning approaches distill into **skills** saved to the registry.
- **Cross-Lab web pipeline:** a **LangGraph meta-graph** connects individual Labs into one end-to-end research pipeline; users compose/inspect it on the **Pipeline Canvas** (React Flow), where each node is a Lab-step, HITL gates sit between stages, and **Evidence flows through** so the whole pipeline is provenance-locked and reproducible end to end.
- **Component/tool/skill registry (plug-in/plug-out):** every capability is a registered **Component** (`connector|transform|chart|model|evaluator|analyzer|decision|report_block|tool|skill`) with a `ComponentSpec` (id, version, JSON-Schema params, ports, tags). `GET /api/components` drives both the agent tool list and the UI (forms render from JSON Schema → **new plugin = zero UI code**). Variants (AR1/AR2) = same id, different params.

Persistence roles (polyglot, dedicated containers)

Store	Role
Postgres	Source of truth for transactional/relational data: orgs/users/RBAC, projects, dataset metadata, runs + reproducibility manifests, **Evidence Ledger**, gate tasks; **pgvector** holds RAG embeddings (`text-embedding-3-small`); **LangGraph checkpointer** (pause→persist→resume).
Neo4j	Graph store: the Paper-Experiment **walkthrough node graph**, **provenance/lineage** (Evidence → run → code → dataset), and paper knowledge graph (variables ↔ models ↔ data sources).
MongoDB	Document store for heterogeneous content: parsed paper sections + structured Paper Card JSON, Signal Lab per-file extraction blocks (pre-consolidation), and agent trace/message logs.
Redis	Celery broker/result backend, cache, SSE/WS pub/sub for live traces, locks.
BlobStore	Datasets, artifacts, generated workbooks/reports → **local volume now**, cloud later.

Memory/grounding: pgvector knowledge base for insights, traces, skills, and **Domain Memory** (org KPIs/definitions) grounding agents; Neo4j for relationship-heavy lineage/knowledge queries.

Monorepo Layout

```
laboratree/
  apps/
    api/ laboratree/
      core/         # settings, db/(postgres,neo4j,mongo,redis clients), security/JWT,
                    #   storage/(BlobStore iface + LocalBlobStore), jobs(Celery+Redis),
                    #   llm/(OpenAI), registry, evidence/(ledger), extract/(multi-format parsers)
      tenancy/      # Org, User, Membership, RBAC, tenant-scoped session
      projects/     # Project, Dataset(versioned/hashed), Run(+repro manifest), Artifact,
                    #   Evidence, GateTask
      agents/       # LangGraph graphs, roles, Docker sandbox executor, checkpointer, interrupt/resume,
                    #   fetch/ (auto data-fetch agent)
      labs/         # signal, paper, data, insight, trend, decision, ideation, collection, intelligence
                    #   modeling/ (families: ml/ dl_pytorch/ econometrics/ timeseries/ anomaly/ + evalu
      pipeline/     # cross-Lab meta-graph (LangGraph) + Pipeline Canvas API (compose Labs end-to-end)
      api/          # routers: auth, orgs, projects, datasets, components, runs, gates, evidence,
                    #   papers, signal, ai
      alembic/
    web/            # Next.js (App Router)
      app/          # auth, workspace, lab pages, paper card, experiment canvas, approval inbox
      components/   # FlowCanvas(React Flow), DropZone(dnd-kit), DynamicForm(JSON Schema),
                    #   ChartRenderer(Vega+ProvenanceBadge), PaperCard, SimplifyControl, AgentTrace,
                    #   GatePanel, ChatPanel
      styles/       # design tokens (light forest theme)
  packages/ plugin-sdk/   # Component/Spec/RunContext + LangGraph tool-adapter contracts
  infra/ docker-compose.yml  # dedicated containers: postgres(+pgvector), redis, neo4j, mongodb,
                             #   + api, worker, web, sandbox image; blobs on a local mounted volume
  CLAUDE.md               # architecture + conventions (build task)
  .claude/skills/laboratree-scaffold/  # skill to scaffold a new Lab/Component from the SDK (build task
  docs/
```

UI / UX (light forest theme)

bg	forest	leaf	sprout	ink
#FBFDF9	#14342A	#6DB33F	#A8D08D	#1E2A22

- **Design tokens:** sidebar/headings in forest, accents/CTAs in leaf. Serif display headings (Lora/Fraunces, echoing the wordmark) + **Inter** for UI/body; generous whitespace; tree/flask motif; rounded organic accents. **Logo + "Grow · Innovate · Impact" branded into report cards & PDF exports.**
- **Drag-and-drop everywhere** — `dnd-kit` for file/component palettes; **React Flow** for the pipeline builder *and* the Paper Experiment walkthrough canvas (drag data/components/processes, wire nodes, fork nodes).
- **ProvenanceBadge** on every figure (links to its Evidence: run + code + output).
- AgentTrace (live agent/tool/code timeline) · GatePanel/Approval Inbox (approve/edit/reject → resumes run) · ChatPanel (streamed OpenAI assist + RAG).

Tooling Deliverables (build tasks)

- `CLAUDE.md` — architecture, module boundaries, registry/plugin conventions, run/agent patterns, theme tokens, how to add a Lab/Component.
- **Claude Code skill** `laboratree-scaffold` — scaffolds a new Lab or Component from the plugin-SDK template (keeps plug-in/plug-out cheap and consistent).

Build Phases

1 Repo & infra

Monorepo, docker-compose with dedicated containers (postgres+pgvector, redis, neo4j, mongodb) + api/worker/web/sandbox; local blob volume; pyproject + Next.js, CI; light-forest design tokens; `CLAUDE.md` seed.

2 Core backbone

DB clients (postgres/neo4j/mongo/redis) + Alembic, `BlobStore` + `LocalBlobStore`, tenancy+auth+RBAC, projects/Run/Artifact/**Evidence**/GateTask + repro manifest, Celery, `plugin-sdk` + registry, `core/llm` OpenAI.

3 Agent runtime + trust layer

LangGraph roles/graph factory, **Postgres checkpointer**, `interrupt` →GateTask→resume, Docker **sandbox executor**, SSE/WS trace; **Evidence Ledger** + **Leakage Sentinel** + reproduce action.

4 Signal Lab

Multi-format extract→reconcile→consolidated master workbook + HITL mapping.

5 Paper Lab (Study)

Paper Card generation, **"explain simpler"** progressive disclosure, chat-with-paper (RAG).

6 Paper Lab (Experiment)

Auto data-fetch agent + HITL fallback; interactive **React Flow walkthrough**; fork-node alternatives + compare-to-paper; real Model Lab components across **ML and DL (PyTorch)** families + Insight/Data components + Red-Team gate powering the nodes.

7 Breadth scaffold + web pipeline

Remaining Labs (Trend/Decision/Ideation/Collection/Intelligence) as interfaces + 1 component + page; the **cross-Lab Pipeline meta-graph** (`pipeline/`) that composes Labs end-to-end with gates + Evidence flow.

8 **Frontend polish**

FlowCanvas, DropZone, DynamicForm, ChartRenderer+ProvenanceBadge, PaperCard, SimplifyControl, AgentTrace, GatePanel, ChatPanel; report/PDF branding with logo.

9 **Tooling**

Finalize `CLAUDE.md` + `laboratree-scaffold` skill.

Verification (end-to-end)

- `docker compose up` brings up all dedicated containers (postgres, redis, neo4j, mongodb) + api/worker/web; `alembic upgrade head` clean; `/health` reports each store connected (incl. Neo4j & Mongo) and the local BlobStore writable; `/api/components` lists all Labs.
- **Tenancy:** org A cannot read org B's data (automated test).
- **Signal Lab:** drop a PDF + 2 CSVs + a DOCX with tables → get one master `.xlsx` with segregated sheets + data dictionary; ambiguous mapping raises a HITL gate.
- **Paper Study:** upload a paper → Paper Card renders all required fields incl. math-with-plain-explanation; "explain simpler" returns a simpler level; chat answers with section citations.
- **Paper Experiment:** auto-fetch retrieves a known-dataset paper's data; for an unavailable one it **stops with the exact link + manual-upload guidance** (no fake success). Walkthrough graph renders for **both an ML paper and a DL (PyTorch) paper**; forking a model node runs an alternative from a *different* family and **compares metrics to the paper**, each carrying a ProvenanceBadge.
- **Web pipeline:** compose a multi-Lab pipeline on the Pipeline Canvas (e.g. Signal → Data → Model → report); it runs end-to-end with a HITL gate between stages and Evidence flowing across Labs.
- **Provenance:** a report claim with no backing Evidence is **blocked**; real metrics link to run/code/output. **Leakage:** injected target-leak feature is flagged. **Reproduce:** re-run a Run → identical metrics from manifest. **Resumability:** kill mid-run at a gate → resumes from checkpoint.
- **Plugin proof:** scaffold a `transform` via the skill → appears in `/api/components`, callable by the agent, form renders with **no frontend changes**. **Sandbox safety:** code runs no-network + resource/time-limited.
- Unit tests: registry, RBAC, gate lifecycle, Evidence integrity, Leakage checks, extraction parsers, each reference component's `run()`.

Explicitly Deferred (post-v1)

> Deepening the scaffolded Labs (full Co-Scientist tournament, Collection synthetic pilots, causal-impact); **exhaustive** model-zoo variant catalog and large-scale GPU training (the ML/DL/econ *families* and representative models ship in v1; filling out every variant comes later); full no-gate autonomy; billing; AutoML leaderboard search; live DB/API connectors; per-tenant BYO API keys; Postgres RLS hardening; multi-document synthesis; skill marketplace. *Each is a new component, agent node, or config — not a rewrite.*
